Smart Library Request Workflow in ServiceNow

ServiceNow Implementation Guide

A comprehensive technical blueprint for implementing a role-based library management system. This document covers automated workflows, security configurations, and reporting dashboards designed to streamline book requests and inventory management on the ServiceNow platform.

Table of Contents

- 1. Problem Statement
- 2. Objectives
- 3. Stakeholders
- 4. Project Phases
- 5. Roles Defined
- 6. Data Architecture
- 7. Flow Designer Automation

- 8. UI Policies
- 9. Access Control Lists (ACLs)
- 10. Step-by-Step Implementation Guide
- 11. Reporting & Dashboard
- 12. Troubleshooting Guide
- 13. Technical Specifications Summary
- 14. Conclusion

1. Problem Statement

In many academic institutions, managing a library system manually creates significant operational challenges. The primary issues include a lack of real-time updates on book availability, inefficient manual approval processes, and limited visibility into borrowing patterns. Students often face difficulties knowing which books are available without physically visiting the library, while librarians struggle with tracking requests, manually approving or rejecting them, and maintaining accurate records of issued or returned books. This disconnect leads to administrative overhead and a poor user experience.

To address these challenges, a role-based Library Management System needs to be implemented on the ServiceNow platform. This system should allow students to request books digitally, while librarians manage the book inventory and approve borrow requests via a centralized dashboard. Automated workflows, role-based access controls, and real-time reports will ensure a streamlined and transparent library management experience.

2. Objectives

- Enable students to submit book borrow requests through a structured digital form.

- Allow librarians to efficiently manage book inventory and approve/reject requests.
- Automate approval workflows and status updates using Flow Designer to reduce manual effort.
- Implement robust role-based access control (ACLs) to ensure data security and privacy.
- Provide reporting dashboards for tracking borrowing patterns and inventory status.

3. Stakeholders

Role	Responsibility
Students	Request books, track request status, and view available catalog.
Librarians	Manage book catalog, approve/reject requests, and generate usage reports.
System Administrators	Configure tables, workflows, roles, and maintain platform health.
Management / Faculty	Use high-level reports for resource planning and acquisition.
IT Support Team	Provide technical support, troubleshooting, and system upgrades.

4. Project Phases

Phase 1: Requirement Analysis & Planning

Define core roles (Student, Librarian), identify data requirements (Book details, Borrow transactions), and map out the workflow automations and security model.

Phase 2: Backend Development & Configurations

Execute role creation in User Administration. Create the foundational tables in System Definition (Book, Borrow Request) and configure dictionary entries (fields).

Phase 3: Development

Build the logic layer: Flow Designer automation setup, UI Policies for form behavior, and Reference Qualifiers to ensure data integrity.

Phase 4: Data Migration, Testing & Security

Define and validate ACLs. Build reports and dashboards. Conduct end-to-end testing and troubleshooting of workflows.

Phase 5: Deployment & Documentation

Final deployment on the PDI (Personal Developer Instance), creation of technical documentation/blueprints, and final presentation preparation.

5. Roles Defined

🎓 Student Role

- **Capabilities:** Can browse the catalog, request books, and track the status of their requests.
- **Access:** Read access to Book records; Create and Read access to their own Borrow Requests only.

📚 Librarian Role

- **Capabilities:** Manages the full book catalog, approves or rejects borrow requests, and generates reports.

- **Access:** Full CRUD permissions (Create, Read, Update, Delete) on all relevant tables.

6. Data Architecture

The system relies on two primary custom tables: `u_book` for inventory and `u_borrow_request` for transaction management.

Table 1: Book Table (u_book)

Field Name	Type	Description
Title	String	Name of the book
Author	String	Author of the book
ISBN	String	Unique book identifier
Status	Choice	**Values:** Available (Default), Issued, Lost

Table 2: Borrow Request Table (u_borrow_request)

Field Name	Type	Description
Requested By	Reference (sys_user)	Auto-populated with logged-in user
Book	Reference (u_book)	The book being requested
Request Date	Date/Time	Auto-filled timestamp
Return Date	Date/Time	Mandatory when Status = Issued
Status	Choice	**Values:** Requested (Default), Approved, Rejected, Returned

> ⚙️ **Technical Note: Reference Qualifier**
> A Reference Qualifier must be applied to the 'Book' field on the Borrow Request table.
> *Filter:* `Status = Available`
> This ensures that students cannot request books that are already issued to others.

7. Flow Designer Automation

The automation logic is built using ServiceNow Flow Designer to handle the approval process and state transitions.

Trigger Configuration

- **Table:** u_borrow_request
- **Condition:** Status changes to "Requested"
- **Event:** Record Created or Updated

Flow Steps

Step	Action	Details
1	Ask for Approval	Request approval from users with the **Librarian** role.
2	If Approved → Update Book Record	Lookup the referenced Book record and set `Status = Issued`.
3	If Approved → Update Borrow Request	Set Borrow Request `Status = Approved`.
4	Send Email Notification	Notify the 'Requested By' user: "Your book has been issued."
5	If Rejected → Update Borrow Request	Set Borrow Request `Status = Rejected`.

Step	Action	Details
6	Return Logic	(Separate Flow/Logic) When Status changes to Returned → Set Book `Status = Available`.

8. UI Policies

UI Policies are used to control the behavior of forms based on data conditions.

UI Policy 1: Lock Fields on Approval

- **Table:** u_borrow_request
- **Condition:** Status is `Approved`
- **Action:** Set *Book* and *Requested By* fields to **Read-Only**. This prevents tampering with the record after approval.

UI Policy 2: Mandatory Return Date

- **Table:** u_borrow_request
- **Condition:** Status is `Issued`
- **Action:** Set *Return Date* field to **Mandatory** and **Visible**. This ensures a return date is captured when the book is physically handed over.

9. Access Control Lists (ACLs)

Security is managed via ACLs to ensure users only access what they are authorized to see.

Book Table (u_book)

Operation	Student	Librarian
Read	✅ Yes	✅ Yes
Write	❌ No	✅ Yes
Create	❌ No	✅ Yes
Delete	❌ No	✅ Yes

Borrow Request Table (u_borrow_request)

Operation	Student	Librarian
Create	✅ Yes	✅ Yes
Read	✅ Yes (Own Records)	✅ Yes (All Records)
Write	❌ No	✅ Yes
Delete	❌ No	✅ Yes

> 🔒 **Security Script:**
> For the Student Read ACL on `u_borrow_request`, use the following condition to restrict access to their own records:
> `javascript: gs.getUserID() == current.u_requested_by`

10. Step-by-Step Implementation Guide

Step 1: Create Roles

1. Navigate to **User Administration > Roles**.
2. Click **New** and create role `librarian`.
3. Repeat to create role `student`.

Step 2: Create Book Table (u_book)

1. Navigate to **System Definition > Tables**.
2. Click **New**. Name: `Book`.
3. Add fields: Title (String), Author (String), ISBN (String), Status (Choice: Available, Issued, Lost).

Step 3: Create Borrow Request Table (u_borrow_request)

1. Navigate to **System Definition > Tables**.
2. Click **New**. Name: `Borrow Request`.
3. Add fields: Requested By (Ref: sys_user), Book (Ref: u_book), Request Date (Date/Time), Return Date (Date/Time), Status (Choice: Requested, Approved, Rejected, Returned).
4. **Important:** Set Reference Qualifier on Book field to `Status=Available`.

Step 4: Configure Flow Designer

1. Navigate to **Flow Designer > New Flow**.
2. Set Trigger: Created/Updated on `u_borrow_request` where `Status = Requested`.
3. Add **Ask for Approval** action assigned to the `librarian` role.
4. Add logic to update record status based on approval outcome.

Step 5: Configure UI Policies

1. Navigate to **System UI > UI Policies**.
2. Create Policy 1: Lock fields when Status is Approved.
3. Create Policy 2: Make Return Date mandatory when Status is Issued.

Step 6: Configure ACLs

1. Navigate to **System Security > Access Control**.
2. Define Read/Write/Create/Delete rules as specified in the ACL section above.

Step 7: Create Reports

1. Navigate to **Reports > Create New**.

2. Create "Most Borrowed Books" report (Bar Chart) grouped by Book.

11. Reporting & Dashboard

Reports allow management and librarians to track KPIs.

Report Name	Type	Table	Configuration
Most Borrowed Books	Bar/Pie Chart	u_borrow_request	Group By: Book | Filter: Status = Approved
Active Borrow Requests	List	u_borrow_request	Filter: Status = Requested
Overdue Returns	List	u_borrow_request	Filter: Return Date < Today

12. Troubleshooting Guide

Issue	Probable Cause	Resolution
Flow not triggering	Trigger condition mismatch	Verify trigger field is "Status" and value is strictly "Requested". Check for any conflicting flows.
Issued books showing in request form	Reference Qualifier missing	Navigate to Dictionary Entry for 'Book' field and ensure Reference Qualifier is set to `Status = Available`.
Return Date not mandatory	UI Policy inactive	Check UI Policy conditions. Ensure "Reverse if false" is configured correctly if needed.
Student seeing all requests	ACL script missing	Verify the Read ACL has the script condition: `gs.getUserID() == current.u_requested_by` .

Issue	Probable Cause	Resolution
Email not sending	Action misconfiguration	Check Flow Designer execution logs. Ensure the "Requested By" user record has a valid email address.

13. Technical Specifications Summary

Component	Technology Used
Platform	ServiceNow (PDI - Personal Developer Instance)
Workflow Engine	Flow Designer
Security Model	Role-based Access Control Lists (ACLs)
UI Configuration	UI Policies, Dictionary Overrides
Reporting	ServiceNow Reports & Dashboards
Data Model	Relational (`u_book` ↔ `u_borrow_request`)

14. Conclusion

The Smart Library Request Workflow in ServiceNow successfully addresses the challenges of manual library management by providing a fully automated, role-based digital system. By leveraging platform capabilities such as Flow Designer, custom tables, and ACLs, the solution ensures that students can seamlessly request books while librarians efficiently manage the catalog and approval process.

The implementation not only digitizes the transaction process but also provides transparency through reporting and ensures data integrity through security policies.

This project serves as a robust blueprint for automating similar resource request workflows within the ServiceNow ecosystem.